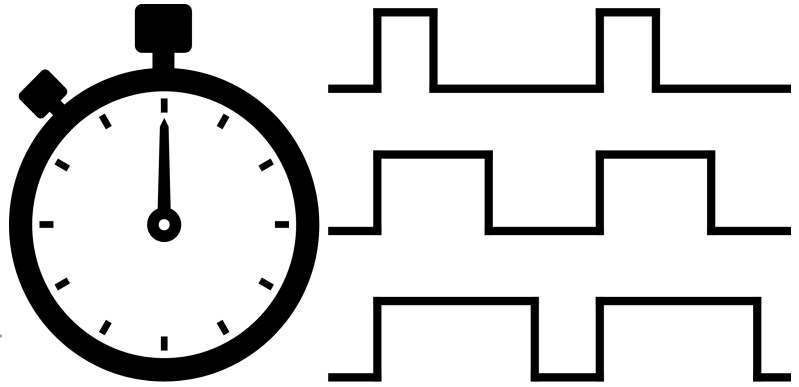


A3 – LCD Display

Supports P1



Reference Materials

- STM32 Reference Manual (RM) – Clock System, GPIO
- FRDM-STM32 – Pin Diagram
- LCD Display / LCD Controller Datasheets

Learning Objectives

1. Develop your ability to to read and understand datasheets
2. Implement software delays
3. Further enhance your mastery of GPIO.

Software Delays

Controlling devices connected to a microprocessor often require specific timing for the control signals. For some devices the control timing can be easily achieved with software delays. In general software delays are not ideal for precise or efficient timing because the processor is kept busy doing nothing and cannot perform other operations while waiting. In some instances like device initialization or printing to a display this is less of an issue because there is nothing else the CPU could do until the necessary time has passed. Future assignments will cover better timing options available to the STM32. Software delays are created by having the processor perform a set number of operations. By controlling how many times this instruction occurs, the wait time can be configured. This is usually achieved with loops of nops (no operation assembly instruction). Adjusting the number of iterations of the loop allows the delay timing to be controlled. This method is intrinsically inaccurate with changing clock frequencies, compiler optimizations, and interrupts. More accurate timing can be achieved using hardware timers.

Delay Function

You may use HAL_Delay() functions.

Instructions

1. Gather and review the datasheet for the LCD display. Be sure to look over the schematic, instruction table, initialization sequence, and write data timing diagram.

Build the Circuit

2. Draw a wiring diagram for connecting the LCD to the STM32 using 8-bit mode to start with. Make sure to label the pins as organized on the STM32 and LCD display. Note that most pins on the STM32 are not organized in order sequentially on the Freedom board. Refer to the STM32 pin diagrams. Be sure to include any necessary power connections. Some LCDs include a V0 pin used to control the contrast on the screen that can be connected to ground through a potentiometer.
3. Construct the circuit. When wiring devices to the STM32 it is important to check all signals connected to the pins. Be careful not to exceed the maximum voltage of the STM32

Code Design

4. Write a short program to initialize the LCD and write a single character to the display. Note that *any time* you write a command or character to the LCD screen, you must make the E signal on the LCD toggle (latching the command or character on the falling edge of the E signal). Upon successful initialization, you should see a blinking cursor on the LCD. Some debug tips:
 - a. If you don't see a cursor, make sure your contrast, power and ground voltages on the LCD module are correct. *note: when debugging digital systems, always check your power/ground and clock as appropriate.*
 - b. Verify that you have wired the bus up correctly
 - c. You may want to put your initialization routine in a large `while()` loop for debug purposes.
5. Modularize your code, writing useful functions to make later use of the LCD display easy to integrate. Minimum functions to be written include:

```
LCD_init(void);           // initialize LCD
LCD_command(uint8_t command); // Send LCD a single 8-bit command
LCD_write_char(uint8_t letter); // write a character to the LCD
```

Note that some #define can be used for specific commands like CLEAR_HOME, NEW_LINE, or CURSOR_BLINK.

Also note that you can combine the LCD_command and LCD_write_char to one LCD_write(uint8_t value, uint8_t mode) where value is then the data or command to write and mode is a single bit to indicate whether you are writing data or a command.

6. Write a program using your functions to print "Hello World" on the top line and "Assignment 3" on the bottom line of the LCD.

Hints

- LCD initialization must be exact. Verify the data bus and enable pulse timing with an oscilloscope or logic analyzer if it is not working.
- Be mindful of the timing for signals going low as well as high. For example, to create 2 pulses, timing delays would need to be added when the signal is set low as well as when set high.
- You may consider writing other functions that will make future use of the LCD easier, such as `LCD_write_string` which could reuse the `LCD_write_char` function to write a string of characters to the LCD.
- The timing specifications in the LCD are minimum specifications - when first developing, you can make the delays 10x normal to help account for any other issues.

Deliverables

1. Demonstrate your working program to the instructor or lab assistant
2. Copy all of your code (`lcd.c`, `lcd.h`, and `main.c`) into a text file - ensure that it is properly formatted (you can copy your code to tohtml.com to help with formatting first) and submit it as a pdf.

**Properly formatted implies properly tabbed with sufficient comments and adjusted so code doesn't wrap, excess white space removed for a compact presentation, following accepted code formatting standards, header comments at the top of each new file to assist the reader in quickly interpreting what they are looking at, etc.*